



Hospital Appointment Management System (HAMS)

Software Design Specification

– Hanoi, July 2026–

RECORD OF CHANGES

Date	A* M, D	In charge	Change Description
25/6	A	DungNV	Added database schema
30/6	A	DungNV	Added table description
09/7	M	DungNV	Modify database schema and table description
15/07	A	DungNV	Added Class diagram for Schedule Management Feature
16/07	A	DungNV	Added Sequence diagram for Create Doctor Schedule use case
19/07	A	DungNV	Added Class diagram for Medical Service Management Feature
19/07	A	HieuNM	Added Class Diagram for Book Appointment Feature

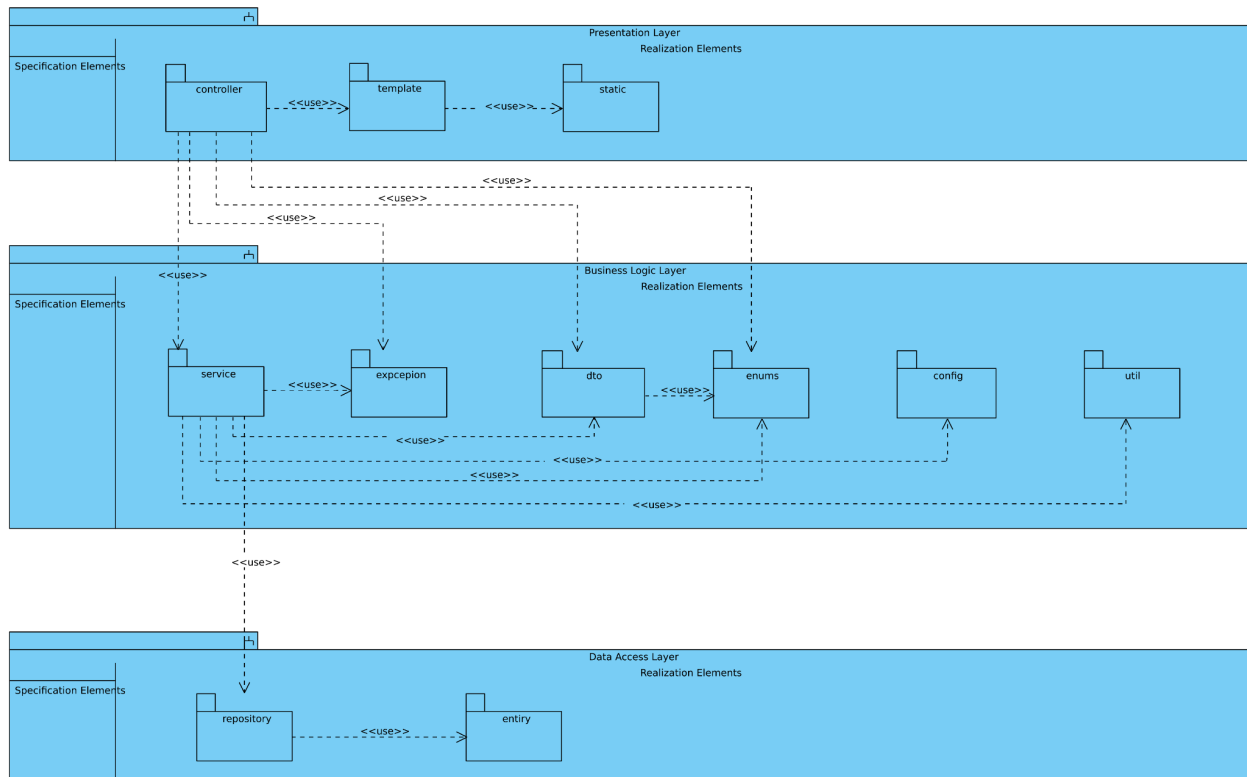
*A - Added M - Modified D - Deleted

Table of Contents

I. Overview.....	4
1. Code Packages.....	4
2. Database Design.....	4
a. Database Schema.....	4
b. Table Description.....	4
II. Code Designs.....	5
1. <Feature/Function Name1>.....	5
a. Class Diagram.....	5
b. Class Specifications.....	5
c. Sequence Diagram(s).....	5
d. Database queries.....	6
2. <Feature/Function Name2>.....	6

I. Overview

1. Code Packages



Package descriptions

No	Package	Description
1	config	Sets up and configures the security settings ,cloudinary, connection and passwordEncode.
2	controller	Receives and handles user requests from the website, split by user roles (admin, manager, doctor, patient, receptionist).
3	dto	Holds data packages (request and response) used to safely transfer data between the client and the server.
4	entity	Represents tables in the database and maps data relationships.
5	enums	Stores a list of fixed constants used across the project (e.g., Appointment Status, User Roles).
6	exception	Handles system errors and returns clean, easy-to-understand error messages to users.
7	repository	Talks directly to the database to save, update, or find data.
8	security	Handles system security, user login (Authentication), and user permissions (Authorization).
9	service	Processes the main business logic and functions of the application.
10	util	Contains common helper tools (date formatting and appointment code generation).
11	templates	Contains HTML files that create the layout and web pages for users to see.

12	static	Stores styling files (CSS) for the website.
----	--------	---

2. Database Design

a. Database Schema

2	roles	User Roles: Defines access levels and system permissions. Stores unique system roles (ADMIN for system settings, MANAGER for hospital schedules, DOCTOR for clinical work, PATIENT for portal clients, and RECEPTIONIST for billing/check-ins) with descriptions.
3	departments	Hospital Departments: Represents specialized departments (e.g., Pediatrics, Cardiology). Stores department name, description, landing page image URL, and active status (ACTIVE/INACTIVE) to determine availability on public-facing portals.
4	rooms	Hospital Rooms: Represents physical consultation or diagnostic rooms. Each room belongs to a specific department (department_id FK), and contains a room name, code, status, and creation/update audit timestamps.
5	users	Unified Accounts: The core table storing credentials (username, email, password hash, status, email verification flags) and common profile data (names, phone, gender, avatar). It houses role-specific fields: date_of_birth/blood_type for Patients, and department_id/license_issue_date/degree/license_number/bio/doctor_status for Doctors.
6	user_roles	User-Role Mappings: A junction table resolving the many-to-many relationship between users and roles. Maps which accounts possess which privileges. Enforces cascading deletes to clean up assignments if a user or role is deleted.
7	user_addresses	User Addresses: Manages mailing or contact addresses for users. Connects a user (user_id) to a province (province_id) with fields for detailed address lines and a default flag (is_default) to mark the primary billing/contact address.
8	medical_services	Medical Service Catalog: Defines clinical offerings (e.g., General Examination, Blood Test, X-Ray) associated with departments. Stores service names, reference pricing, estimated duration in minutes (defaults to 30), descriptions, media URLs, and active statuses.

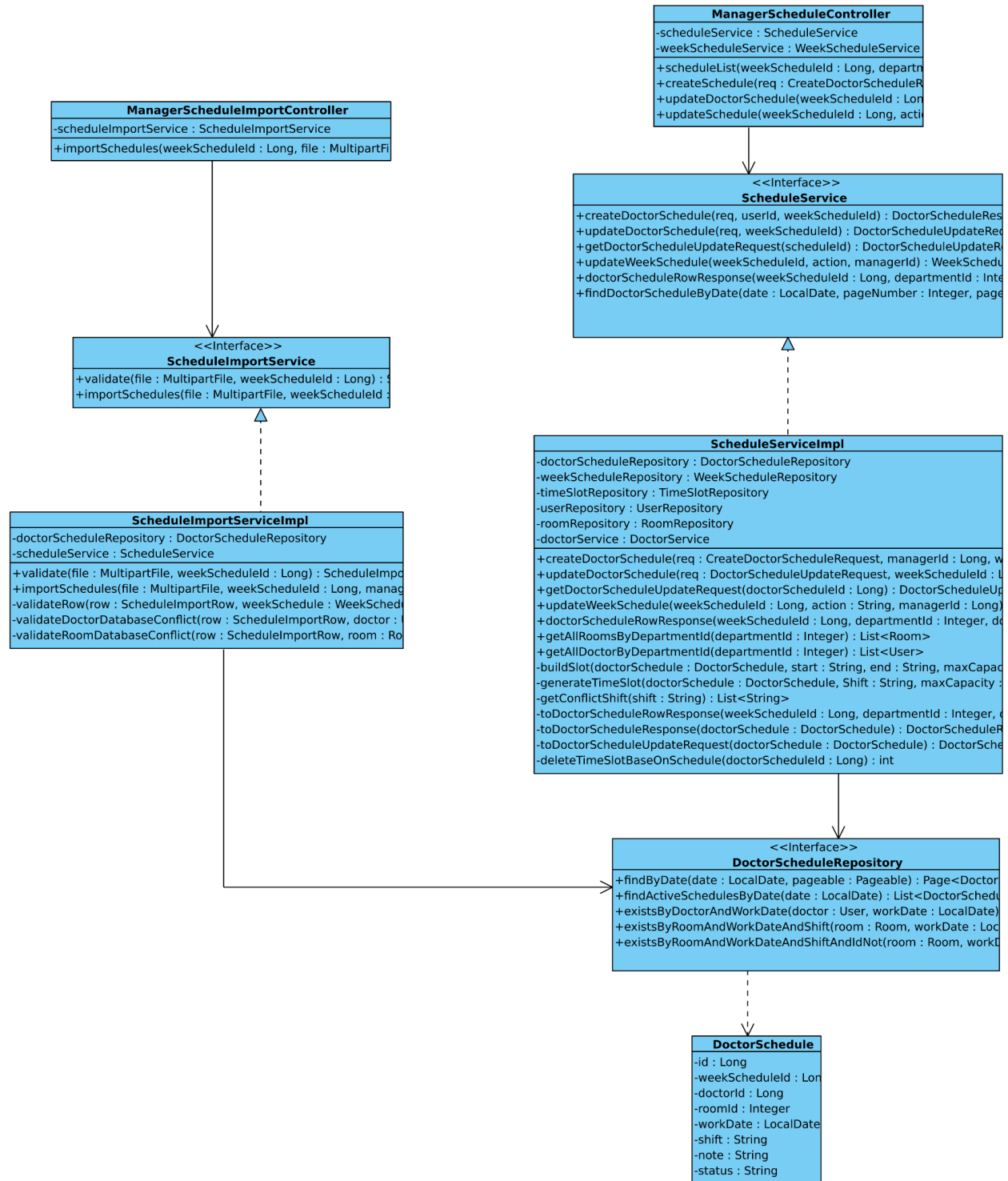
9	week_schedules	Weekly Hospital Schedules: Controls the scheduling lifecycle on a week-by-week basis. The schedule covers Monday to Sunday and moves through statuses: DRAFT (manager planning), PUBLISHED (visible to doctors), FINALIZED (bookable by patients), or EXPIRED (due date passed without finalized release). It enforces a unique constraint on the week start date.
10	doctor_schedules	Doctor Work Shifts: Links doctors to specific rooms on calendar dates under a weekly schedule sheet (week_schedule_id). Defines shift times (MORNING, AFTERNOON, FULL_DAY), manager notes, and statuses (ACTIVE or CANCELLED).
11	time_slots	Appointment Time Slots: Sub-divided booking windows within a doctor's work shift (e.g., 07:00-09:00, 09:00-12:00). Tracks current occupancy (booked_capacity), max limit (max_capacity), availability status (AVAILABLE/FULL), and an optimistic locking version count to handle concurrent bookings safely.
12	appointments	Patient Bookings: Records consultation bookings made by patients. Stores booking codes, patient, doctor, and slot IDs, check-in timestamps, and a status workflow: PENDING (booking pending), CONFIRMED (confirmed), IN_PROGRESS (check-in complete and paid), COMPLETED (examination done), or CANCELLED (canceled booking).
13	medical_records	Patient Medical Records: Stores clinical outcomes and diagnosis documents (strict 1-1 relationship with appointments). Contains vital signs (heart rate, blood pressure, blood glucose, weight), symptoms, primary diagnosis, conclusion, prescriptions, notes, and records status (DRAFT vs. FINALIZED).
14	medical_service_orders	Diagnostic Test Orders: Tracks lab tests, X-rays, or specialized procedures ordered by doctors during a consultation. Records the service price at the time of ordering, results, and order statuses: PENDING_PAYMENT (not paid), PAID (paid, ready for execution), COMPLETED (results uploaded by doctors), or CANCELLED (refunded/dismissed).
15	invoices	Billing Invoices: Tracks payments made for appointments (allows 1-to-N invoices per appointment: e.g., Invoice #1 for clinical check-in fee, Invoice

		#2+ for diagnostic tests ordered during the visit). Tracks billing code, total amount, payment method, payment status (UNPAID, PAID, REFUNDED), and timestamp.
16	invoice_items	Invoice Line Items: Lists itemized lines for services charged inside an invoice. Includes applied price, quantity, and line totals. It links to medical_service_orders via a nullable key (NULL for check-in exams, NOT NULL for diagnostic orders), protecting unique order-to-invoice relationships with a filtered index.
17	articles	Articles: Stores news, blogs, and public health articles. Combines news and knowledge base contents by using a category field. Tracks title, SEO slug, content summary, authoring doctor, creator, view count, publishing status (DRAFT/PUBLISHED), and release dates.
18	email_logs	Transactional Email Logs: Audits system-sent emails (booking confirmations, verification links, alerts). Records email type, recipient address, email subject, send status, exception messages, and polymorphic relation keys (related_entity_id, related_entity_type) to link logs to specific system objects.
19	notifications	In-App Notifications: Stores notifications shown to users on their web or mobile dashboards. Holds the user ID, title, rich message body, notification category, read/unread flag, and polymorphic references for interactive deep-linking.
20	article_comments	Article Comments: Stores comments and nested replies posted by users on health articles. Holds the article ID, user ID, comment content, creation timestamp, and self-referencing parent ID to support multi-level discussion threads.

II. Code Designs

1. Feature Schedule Management - Manager

a. Class Diagram



b. Class Specifications

ManagerScheduleController Class Handles HTTP requests from Managers for viewing, creating, and updating doctor schedules manually.

ManagerScheduleController

No	Method	Description
1	scheduleList	Inputs: weekScheduleId, departmentId, doctorName, shift, page, size, model Outputs: String (View template name) Processing: Gets a list of doctor schedules based on filters like week, department, and shift. Return this list to the web page.
2	createSchedule	Inputs: req (CreateDoctorScheduleRequest), weekScheduleId, redirectAttributes Outputs: String (Redirect URL) Processing: Receives data from the form to make a new schedule. Checks if the data is correct, calls the service to save it, and shows a success or error message.
3	updateDoctorSchedule	Inputs: weekScheduleId, req (DoctorScheduleUpdateRequest), filters, bindingResult, redirectAttributes Outputs: String (Redirect URL) Processing: Updates an existing doctor schedule. Check if the room or doctor is already busy. If everything is okay, it saves the changes and reloads the page.
4	updateSchedule	Inputs: weekScheduleId, action (publish/finalize), filters, redirectAttributes Outputs: String (Redirect URL) Processing: Changes the status of the whole week schedule (for example, changes from DRAFT to PUBLISHED).

ManagerScheduleImportController Class: Handles the uploading of Excel files to import schedules.

ManagerScheduleImportController Class

Class Methods

No	Method	Description
1	importSchedules	Inputs: weekScheduleId, file (MultipartFile), redirectAttributes Outputs: String (Redirect URL) Processing: Takes the uploaded Excel file (.xlsx), finds out who the manager is, and sends the file to the service to process. Returns how many rows were added or shows error messages.

ScheduleService Interface Defines the main rules for managing schedules.

ScheduleService Interface

No	Method	Description
01	createDoctorSchedule	Inputs: req, userId, weekScheduleId Outputs: DoctorScheduleResponse Processing: A rule to check if rooms and doctors are free, and then save a new schedule with its time slots.
02	updateDoctorSchedule	Inputs: req, weekScheduleId Outputs: DoctorScheduleUpdateRequest Processing: A rule to update a schedule. It also creates new time slots if the working shift changes.
03	updateWeekSchedule	Inputs: weekScheduleId, action, managerId Outputs: WeekSchedule Processing: A rule to change the status of a week schedule and send emails to doctors.

ScheduleImportService Interface Defines the rules for reading and saving schedules from Excel files.

ScheduleImportService

No	Method	Description
01	validate	Inputs: file (MultipartFile), weekScheduleId Outputs: ScheduleImportResult Processing: A rule to check if the data in the Excel file is correct, without saving it to the database.
02	importSchedules	Inputs: file (MultipartFile), weekScheduleId, managerId Outputs: ScheduleImportResult Processing: A rule to check the Excel file again and save the good rows into the database.

ScheduleServiceImpl Class Does the real work for managing schedules and time slots.

ScheduleServiceImpl

No	Method	Description
01	createDoctorSchedule	Inputs: req, managerId, weekScheduleId Outputs: DoctorScheduleResponse Processing: Checks if the doctor and room are free. Saves a new DoctorSchedule and creates 2 to 4 TimeSlot parts based on the shift (MORNING or AFTERNOON).
02	updateDoctorSchedule	Inputs: req, weekScheduleId Outputs: DoctorScheduleUpdateRequest Processing: Checks for new problems. If the shift changes, it deletes old time slots and makes new ones. If the schedule is cancelled, it cancels all patient appointments for that time.
03	updateWeekSchedule	Inputs: weekScheduleId, action, managerId Outputs: WeekSchedule Processing: Changes the week status. Sends emails and system messages to doctors when their schedules are ready.
04	generateTimeSlot	Inputs: doctorSchedule, shift, maxCapacity Outputs: List<TimeSlot> Processing: A helper method. It calculates the start and end time for each shift and creates the TimeSlot objects.
05	deleteTimeSlotBaseOnSchedule	Inputs: doctorScheduleId Outputs: int (deleted count) Processing: A helper method. It deletes all time slots of a schedule before making new ones.

ScheduleImportServiceImpl Class Does the real work for reading Excel files and saving them.

ScheduleImportServiceImpl

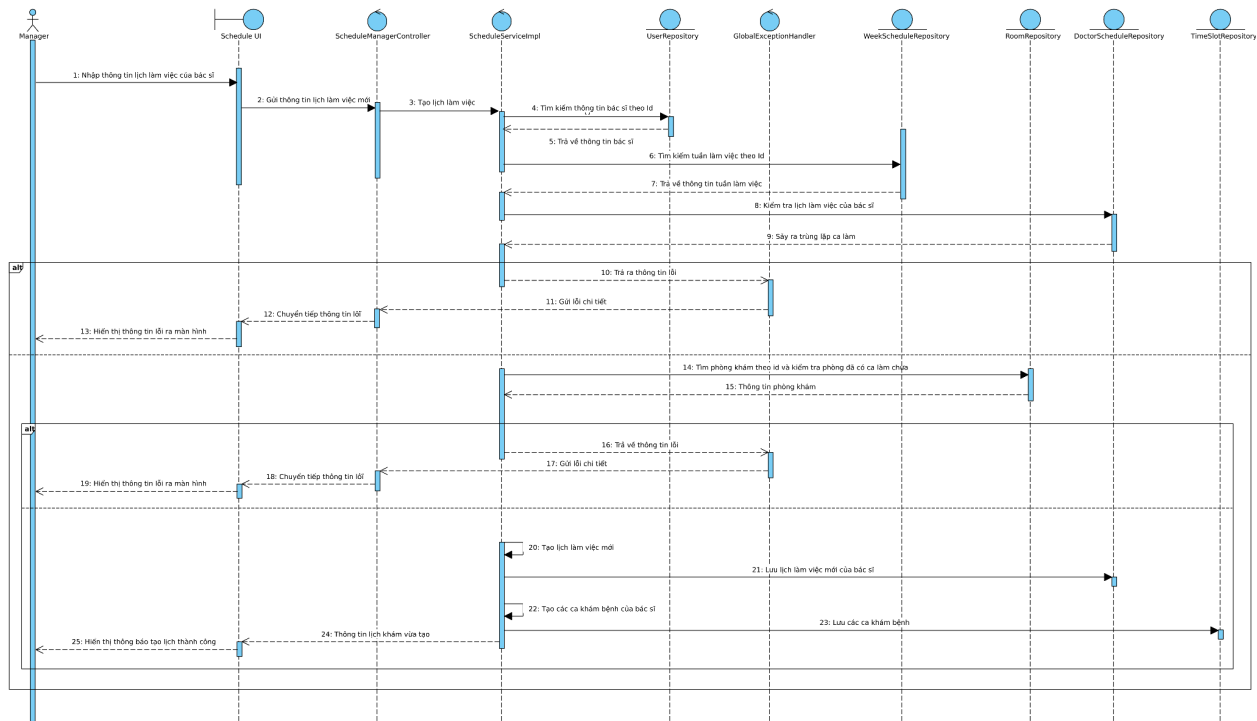
No	Method	Description
01	validate	Inputs: file, weekScheduleId Outputs: ScheduleImportResult Processing: Reads the Excel file. Checks if dates and numbers are correct. It also checks if a doctor or room appears twice at the same time in the file. Returns a list of errors.
02	importSchedules	Inputs: file, weekScheduleId, managerId Outputs: ScheduleImportResult Processing: Runs the validate() method first. If there are no errors, it loops through the file and calls ScheduleService.createDoctorSchedule() to save them.
03	validateRow	Inputs: row, weekSchedule, errors Outputs: void Processing: A helper method. It checks one single row to see if the date, room, and doctor are valid.
04	validateDoctorDatabaseConflict	Inputs: row, doctor, errors Outputs: void Processing: A helper method. It asks the database if the doctor is already busy on that day.
05	validateRoomDatabaseConflict	Inputs: row, room, errors Outputs: void Processing: A helper method. It asks the database if the room is already used on that day and shift.

DoctorScheduleRepository Interface Connects to the database to find and save DoctorSchedules.

DoctorScheduleRepository

No	Method	Description
01	findByDate	Inputs: date, pageable Outputs: Page<DoctorSchedule> Processing: Runs a query to get a list of schedules for one specific day, including doctor and department information.
02	existsByDoctorAndWork Date	Inputs: doctor, workDate Outputs: boolean Processing: Runs a query to check if a doctor already has a schedule saved for a specific day.
03	existsByRoomAndWorkDateAndShift	Inputs: room, workDate, shifts Outputs: boolean Processing: Runs a query to check if a room is already used on a specific day and shift.

c. Sequence Diagram(s)

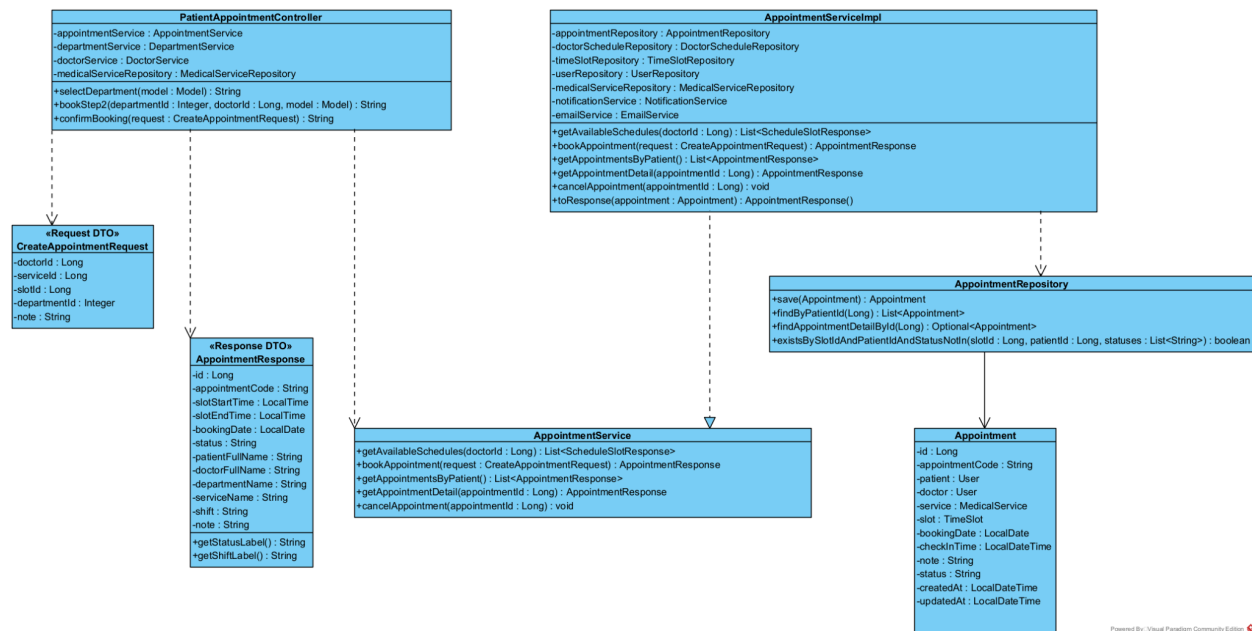


d. Database Queries

[Provide the detailed SQL (select, insert, update...) which are used in implementing the function/screen]

2. Feature Patient/guest

a. Class Diagram Book Appointment



b. Class Specifications

PatientAppointmentController Class : Handles HTTP requests from Patients for viewing, selecting departments/doctors, and booking appointments.

PatientAppointmentController

No	Method	Description
01	selectDepartment	Inputs: model Outputs: String (View template name) Processing: Gets a list of all active departments from the service and sends it to the web page for the patient to choose.
02	bookStep2	Inputs: departmentId (Integer), doctorId (Long), model Outputs: String (View template name) Processing: Gets the selected department and the list of doctors in that department. If a doctor is selected, it calls the service to get the available time slots for that doctor and converts them to JSON for the web page to display.
03	confirmBooking	Inputs: request (CreateAppointmentRequest), bindingResult, userDetails (CustomUserDetails), redirectAttributes Outputs: String (Redirect URL) Processing: Receives the booking form data. Checks if the data is correct. If no service is selected, it automatically finds the default "general examination" service for that department. Then calls the service to save the new appointment and shows a success or error message.

04	cancelAppointment	Inputs: id (Long), userDetails (CustomUserDetails), redirectAttributes Outputs: String (Redirect URL) Processing: Receives the appointment ID from the patient. Calls the service to change the appointment status to CANCELLED and shows a result message.
----	-------------------	--

AppointmentService Interface

Defines the main rules for managing patient appointments.

AppointmentService

No	Method	Description
01	getAvailableSchedules	Inputs: doctorId (Long) Outputs: List<ScheduleSlotResponse> Processing: A rule to get all active time slots of a doctor from today onward. Filters out slots that are already full and organizes them by date and shift.
02	bookAppointment	Inputs: patientId (Long), request (CreateAppointmentRequest) Outputs: AppointmentResponse Processing: A rule to validate the slot availability, check for duplicate bookings, then save a new Appointment record and update the slot's booked capacity.
03	getAppointmentsByPatient	Inputs: patientId (Long) Outputs: List<AppointmentResponse> Processing: A rule to get all appointment records that belong to a specific patient, ordered by creation date.
04	getAppointmentDetail	Inputs: appointmentId (Long), patientId (Long) Outputs: AppointmentResponse Processing: A rule to get the full detail of one appointment and verify that the appointment belongs to the requesting patient.
05	cancelAppointment	Inputs: appointmentId (Long), patientId (Long) Outputs: void Processing: A rule to verify ownership of the appointment, change its status to CANCELLED, release the slot capacity, and send a cancellation notification and email to the patient.

AppointmentServiceImpl Class

Does the real work for managing appointments, time slots, and sending notifications.

AppointmentServiceImpl

No	Method	Description
01	getAvailableSchedules	Inputs: doctorId (Long)

		Outputs: List<ScheduleSlotResponse> Processing: Calls DoctorScheduleRepository to get all ACTIVE schedules of the doctor from today. For each schedule, gets the time slots from TimeSlotRepository and filters them. For today's date, only shows slots that have not started yet. Returns the final list grouped by schedule.
02	bookAppointment	Inputs: patientId (Long), request (CreateAppointmentRequest) Outputs: AppointmentResponse Processing: Finds the patient, doctor, service, and slot by their IDs. Checks if the slot status is AVAILABLE and if the patient has not already booked the same slot. Increases the slot's bookedCapacity by 1. If the slot is now full, changes its status to FULL. Saves a new Appointment with a generated code and status CONFIRMED.
03	getAppointmentsByPatient	Inputs: patientId (Long) Outputs: List<AppointmentResponse> Processing: Calls AppointmentRepository to get all appointment records for the patient with full details (slot, schedule, room, doctor, service) using a JOIN FETCH query to avoid lazy loading errors. Maps each result to AppointmentResponse.
04	getAppointmentDetail	Inputs: appointmentId (Long), patientId (Long) Outputs: AppointmentResponse Processing: Gets one appointment by its ID. Checks that the patient ID on the record matches the given patientId. If they do not match, throws a BadRequestException. Returns the full mapped response.
05	cancelAppointment	Inputs: appointmentId (Long), patientId (Long) Outputs: void Processing: Finds the appointment and checks ownership. Checks if the status is WAITING or CONFIRMED before allowing cancellation. Decreases the slot's bookedCapacity by 1 and sets its status back to AVAILABLE. Sets the appointment status to CANCELLED. Sends an in-app notification and an email to the patient with cancellation details.
06	toResponse	Inputs: a (Appointment) Outputs: AppointmentResponse Processing: A helper method that maps all fields from an Appointment entity to an AppointmentResponse DTO, including patient info, doctor info, slot time, room number, shift, and service name.

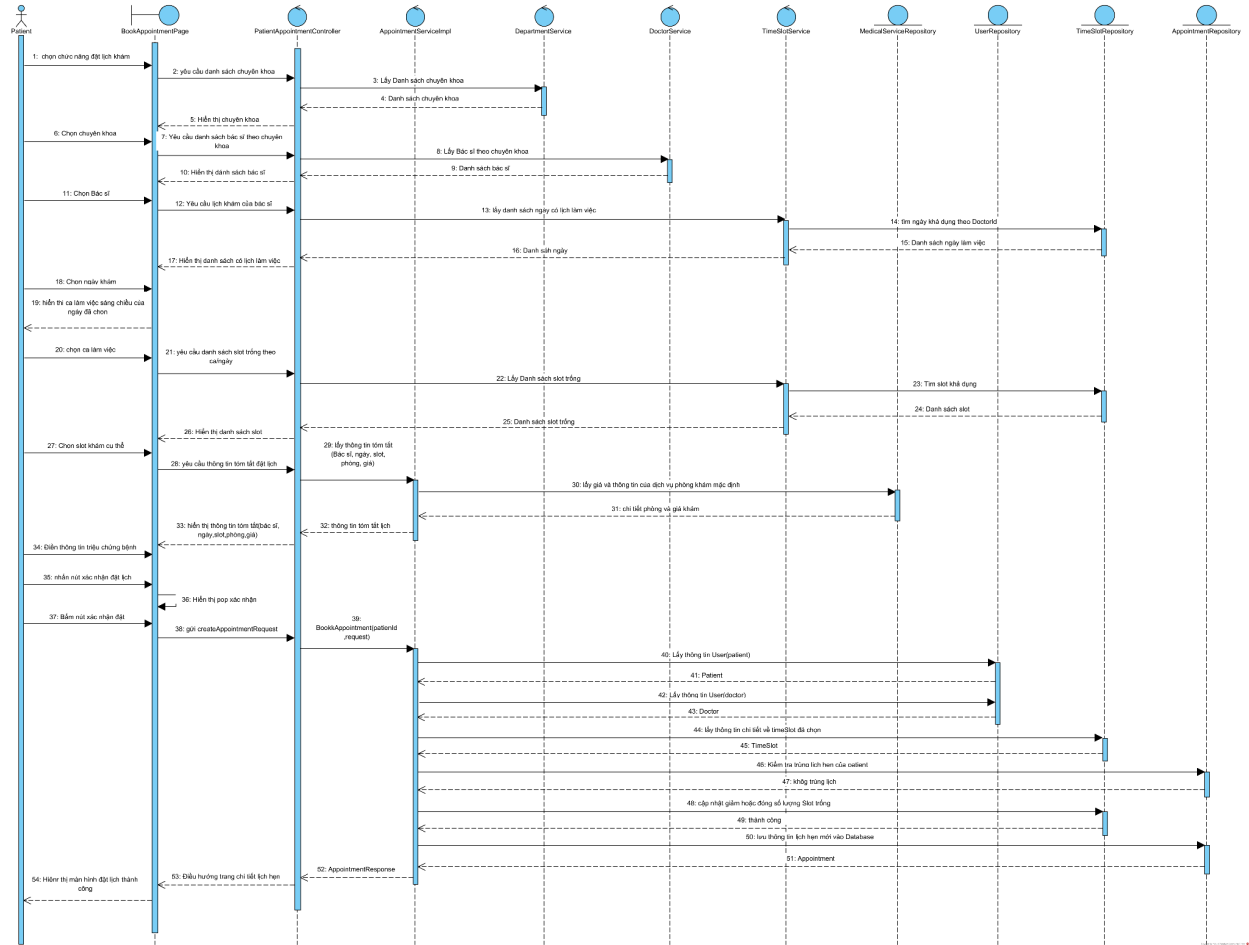
AppointmentRepository Interface

Connects to the database to find and save Appointment records.

AppointmentRepository

No	Method	Description
01	findByPatientIdWith Details	Inputs: patientId (Long) Outputs: List<Appointment> Processing: Runs a JPQL query with JOIN FETCH to get all appointments for a patient including their slot, schedule, room, doctor, department, and service data in one query. Orders by creation date descending.
02	existsBySlotIdAndPatientIdAndStatusNotIn	Inputs: slotId (Long), patientId (Long), excludedStatuses (List<String>) Outputs: boolean Processing: Runs a query to check if the patient already has an active booking for the same slot, ignoring records with CANCELLED or NO_SHOW status. Returns true if a duplicate exists.
03	findAppointmentDetailById	Inputs: appointmentId (Long) Outputs: Optional<Appointment> Processing: Runs a JPQL query with JOIN FETCH to get one appointment with all related data including patient address, province, doctor, service, department, slot, schedule, and room.

c. Sequence Diagram(s)

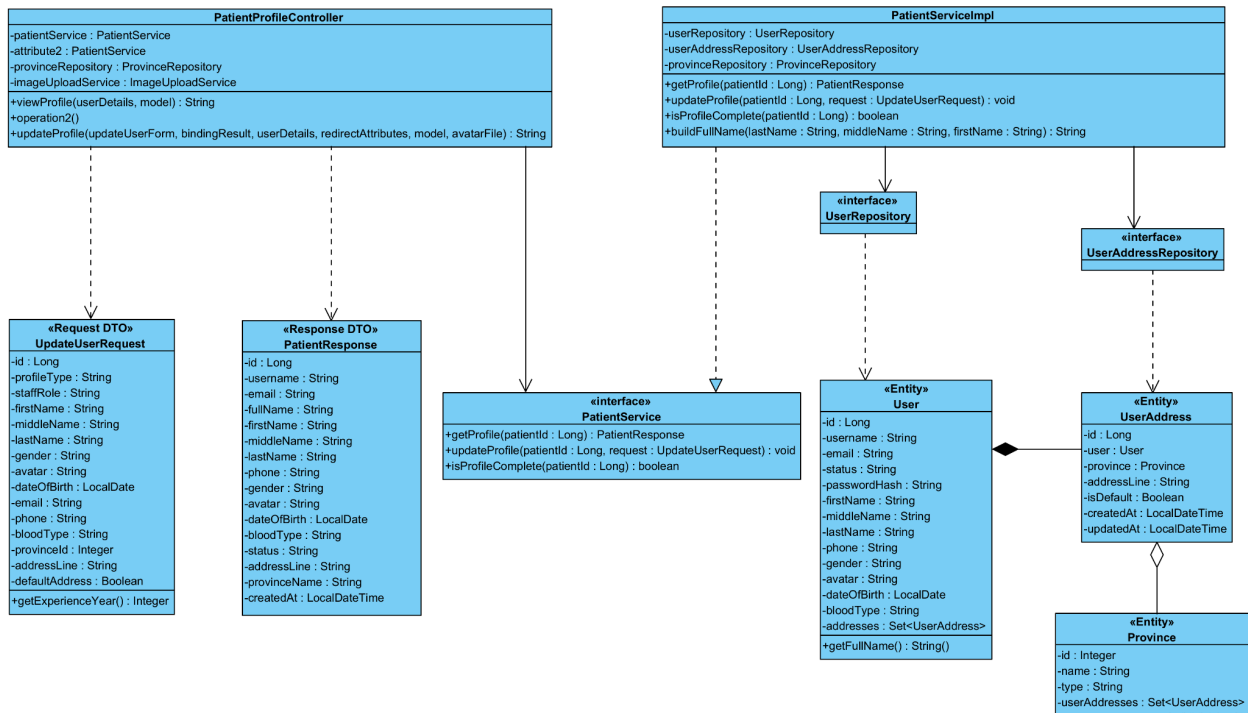


d. Database Queries

[Provide the detailed SQL (select, insert, update...) which are used in implementing the function/screen]

2.1 Feature Patient/guest

a. Class Diagram Book Appointment



b. Class Specifications

PatientProfileController Class

Handles web requests from patients to view, edit, and update their personal profiles, including uploading their avatar.

PatientProfileController

No	Method	Description
01	viewProfile	Inputs: userDetails (CustomUserDetails), model (Model) Outputs: String (template path: "patient/profile/detail") Processing: Retrieves the patient ID from the authenticated user's information (userDetails), invokes patientService.getProfile() to obtain the patient's profile information, and adds it to the model for displaying on the profile detail page.
02	editProfileForm	Inputs: userDetails (CustomUserDetails), model (Model) Outputs: String (template path: "patient/profile/edit") Processing: Retrieves the patient ID, calls staffService.getPatientToUpdate() to prepare the profile update form, and loads the list of provinces from provinceRepository into the model for displaying the edit profile form.
03	updateProfile	Inputs: updateUserForm (UpdateUserRequest), bindingResult (BindingResult), userDetails (CustomUserDetails), redirectAttributes (RedirectAttributes), model (Model), avatarFile (MultipartFile) Outputs: String (redirect URL or template path)

		Processing: Sets the profile type to PATIENT , validates the submitted data, and if valid, uploads the avatar image through imageUploadService, updates the patient's profile via patientService.updateProfile(), and redirects the user to the profile detail page with a success message.
--	--	---

PatientService Interface

Defines contract methods for patient profile management and medical record retrieval.

PatientService

No	Method	Description
01	getProfile	Inputs: patientId (Long) Outputs: PatientResponse Processing: Defines the method for retrieving the patient's personal profile information together with the default address.
02	updateProfile	Inputs: patientId (Long), request (UpdateUserRequest) Outputs: void Processing: Defines the method for updating the patient's personal information and default address information.
03	isProfileComplete	Inputs: patientId (Long) Outputs: boolean Processing: Defines the method for checking whether the patient has completed all required personal profile information.
04	getMedicalRecords	Inputs: patientId (Long) Outputs: List<MedicalRecordResponse> Processing: Defines the method for retrieving all completed medical records belonging to the patient.
05	getMedicalRecordDetail	Inputs: recordId (Long), patientId (Long) Outputs: MedicalRecordResponse Processing: Defines the method for retrieving the details of a specific medical record that belongs to the patient.

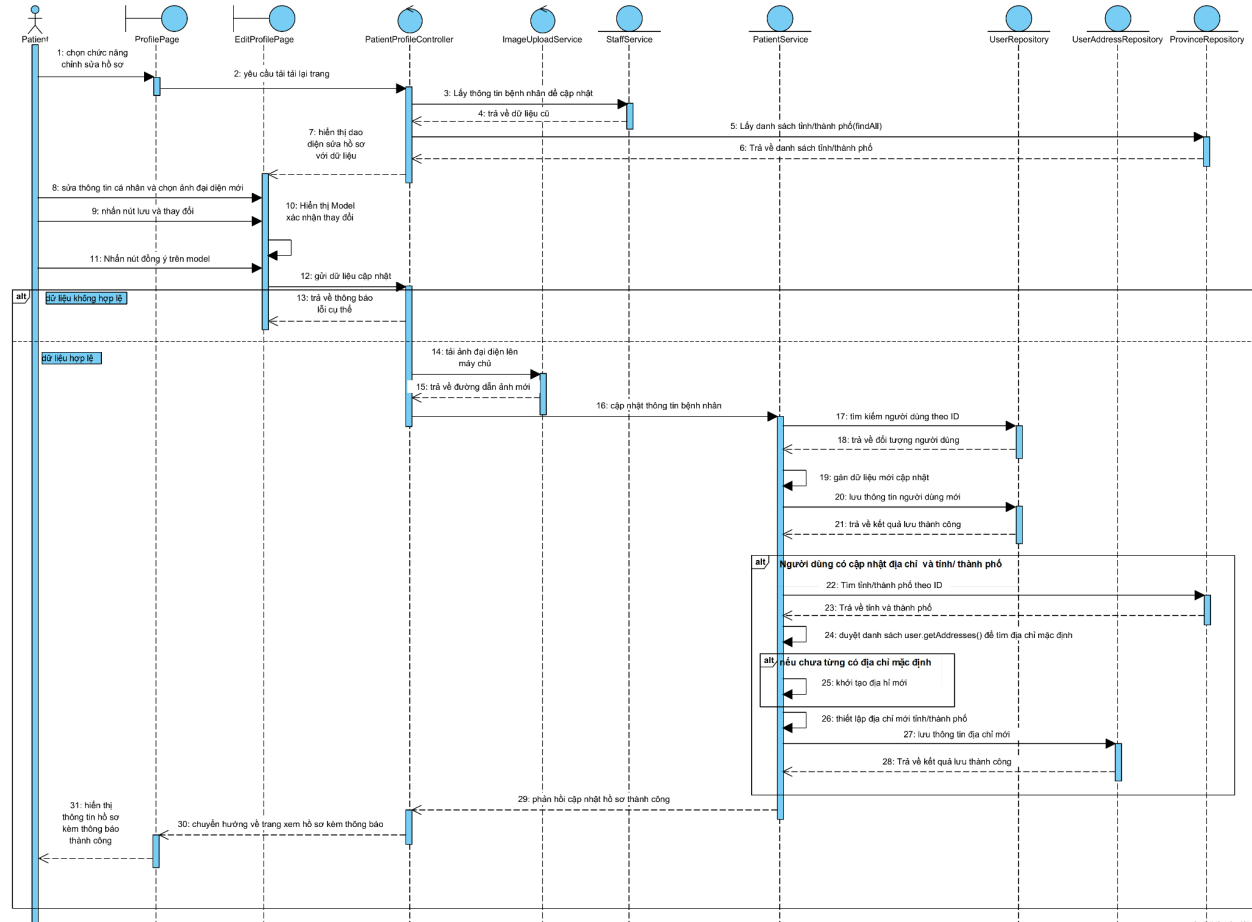
PatientServiceImpl Class

Implements patient profile management, profile validation, database updates, and entity-to-DTO mapping.

PatientServiceImpl

No	Method	Description
01	getProfile	Inputs: patientId (Long) Outputs: PatientResponse Processing: Retrieves the patient from UserRepository using the patient ID, obtains the patient's default address from the address list, maps the retrieved data into a PatientResponse object, and returns the completed profile information.
02	updateProfile	Inputs: patientId (Long), request (UpdateUserRequest) Outputs: void Processing: Updates the patient's personal information, including full name, phone number, gender, date of birth, blood type, and avatar. If address information is provided, validates the selected province using ProvinceRepository, then updates or creates the default address record in UserAddressRepository.
03	isProfileComplete	Inputs: patientId (Long) Outputs: boolean Processing: Checks whether the patient's profile contains all mandatory information required for appointment booking. The required information includes first name, last name, phone number, email address, gender, date of birth, and at least one valid default address containing both province and detailed address. Returns true if the profile is complete; otherwise returns false .
04	getMedicalRecords	Inputs: patientId (Long) Outputs: List<MedicalRecordResponse> Processing: Retrieves all medical records associated with the patient from MedicalRecordRepository, filters only those with the FINALIZED status, converts them into MedicalRecordResponse DTOs, and returns the resulting list.
05	getMedicalRecordDetail	Inputs: recordId (Long), patientId (Long) Outputs: MedicalRecordResponse Processing: Retrieves the specified medical record by its ID, verifies that the record belongs to the patient and has the FINALIZED status, retrieves any additional examination service details if applicable, maps the result into a MedicalRecordResponse object, and returns it.

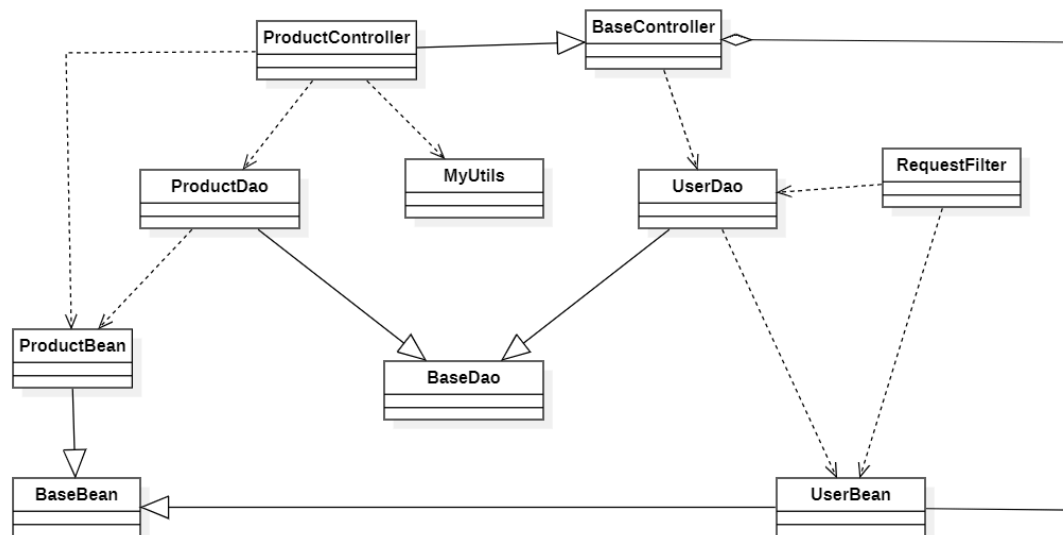
c. Sequence Diagram(s)



3. Feature Manager/Function Create Schedule

a. Class Diagram

[This part presents the class diagram for the relevant feature]



b. Class Specifications

[Provide the description for each class and the methods in each class, following the table format as below]

XYZ Class

[Provide the detailed description for the class methods]

No	Method	Description
01	<method name>	<Description of the method, including the inputs, outputs & internal method processing>

ABC Class

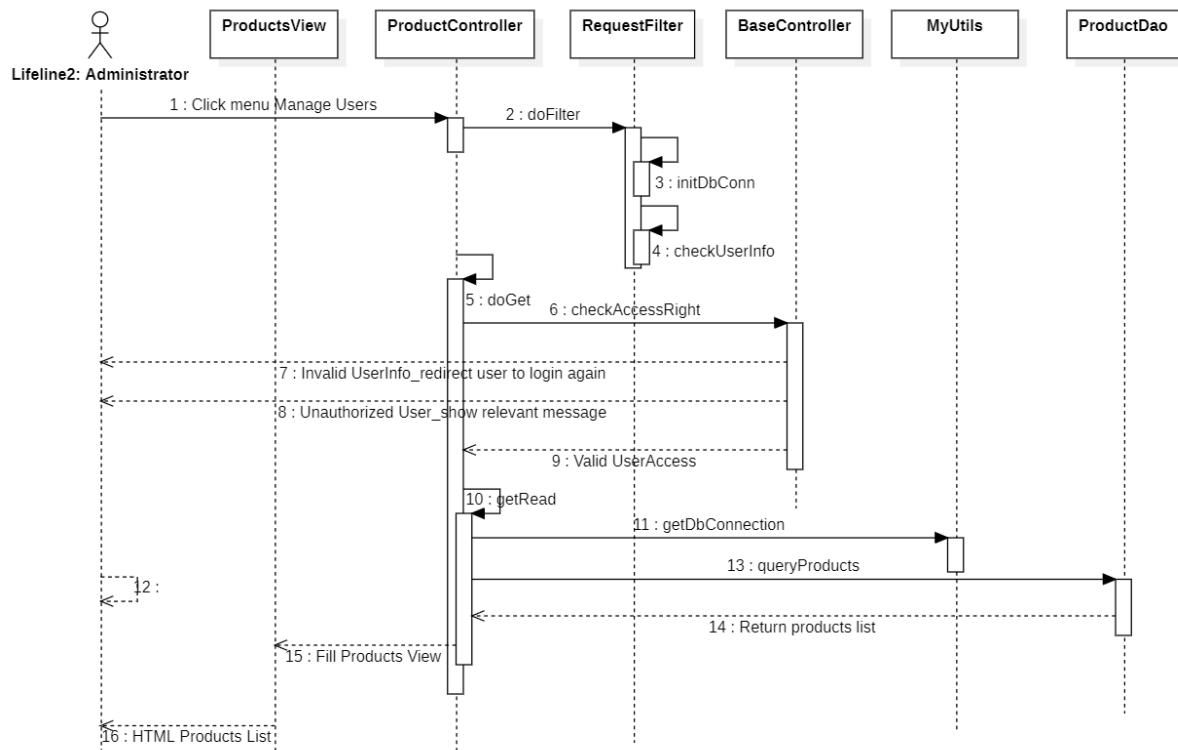
Class Methods

[Provide the detailed description for the class methods]

No	Method	Description
01	<method name>	<Description of the method, including the inputs, outputs & internal method processing>

c. Sequence Diagram(s)

[Provide the sequence diagram(s) for the feature, see the sample below]



d. Database Queries

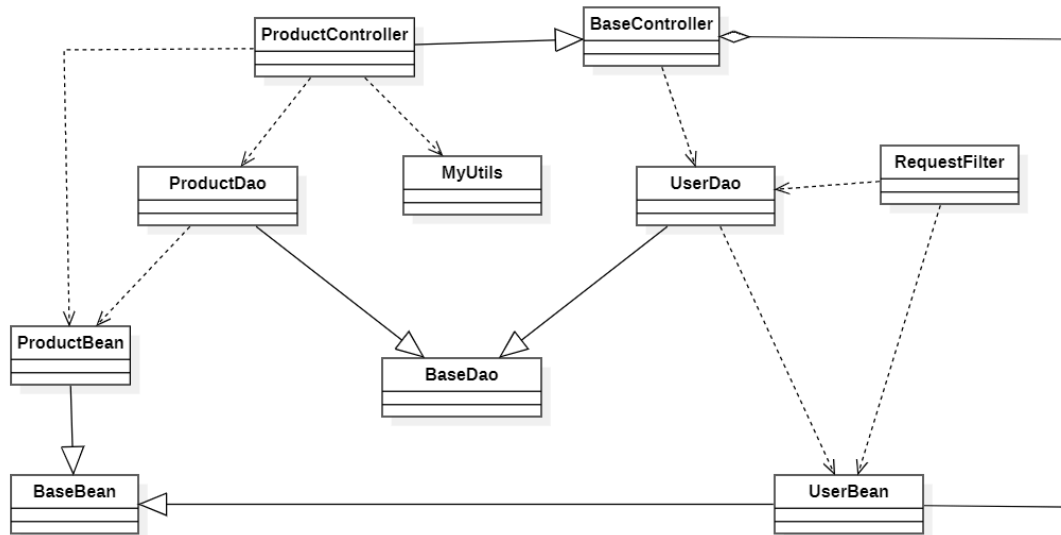
[Provide the detailed SQL (select, insert, update...) which are used in implementing the function/screen]

4. <Feature Receptionist/Function Name1>

[Provide the detailed design for the function <Feature/Function Name1>. It include Class Diagram, Class Specifications, and Sequence Diagram(s)]

a. Class Diagram

[This part presents the class diagram for the relevant feature]



b. Class Specifications

[Provide the description for each class and the methods in each class, following the table format as below]

XYZ Class

[Provide the detailed description for the class methods]

No	Method	Description
01	<method name>	<Description of the method, including the inputs, outputs & internal method processing>

ABC Class

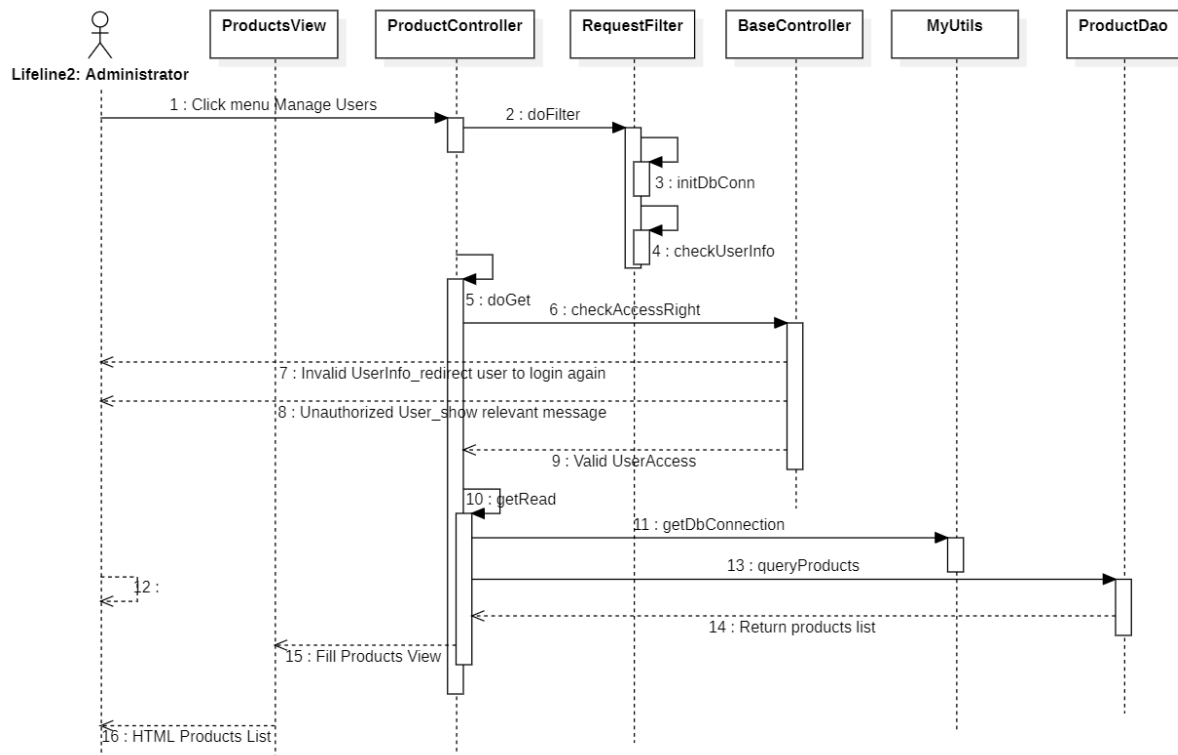
Class Methods

[Provide the detailed description for the class methods]

No	Method	Description
01	<method name>	<Description of the method, including the inputs, outputs & internal method processing>

c. Sequence Diagram(s)

[Provide the sequence diagram(s) for the feature, see the sample below]



d. Database Queries

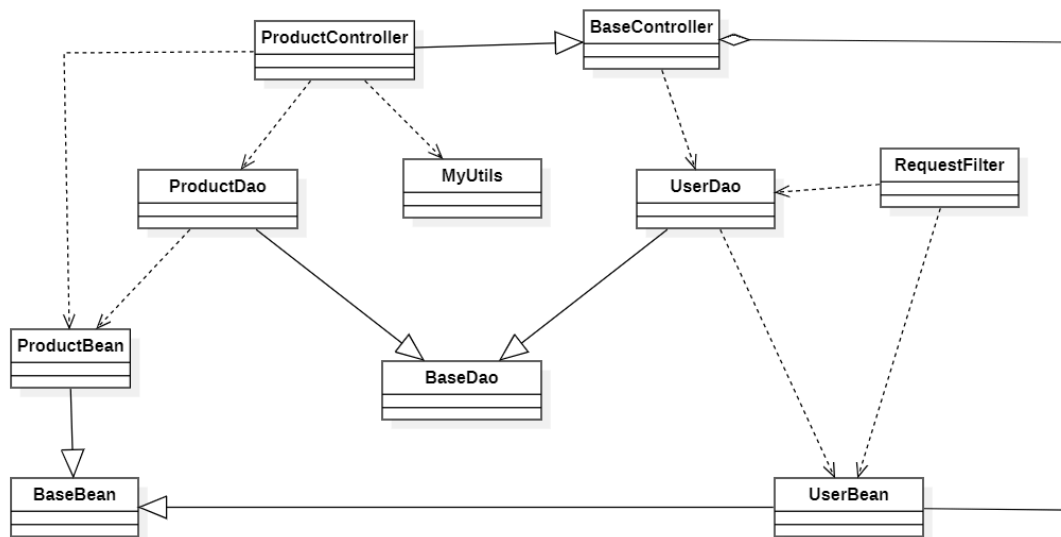
[Provide the detailed SQL (select, insert, update...) which are used in implementing the function/screen]

5. <Feature admin/Function Name1>

[Provide the detailed design for the function <Feature/Function Name1>. It include Class Diagram, Class Specifications, and Sequence Diagram(s)]

a. Class Diagram

[This part presents the class diagram for the relevant feature]



b. Class Specifications

[Provide the description for each class and the methods in each class, following the table format as below]

XYZ Class

[Provide the detailed description for the class methods]

No	Method	Description
01	<method name>	<Description of the method, including the inputs, outputs & internal method processing>

ABC Class

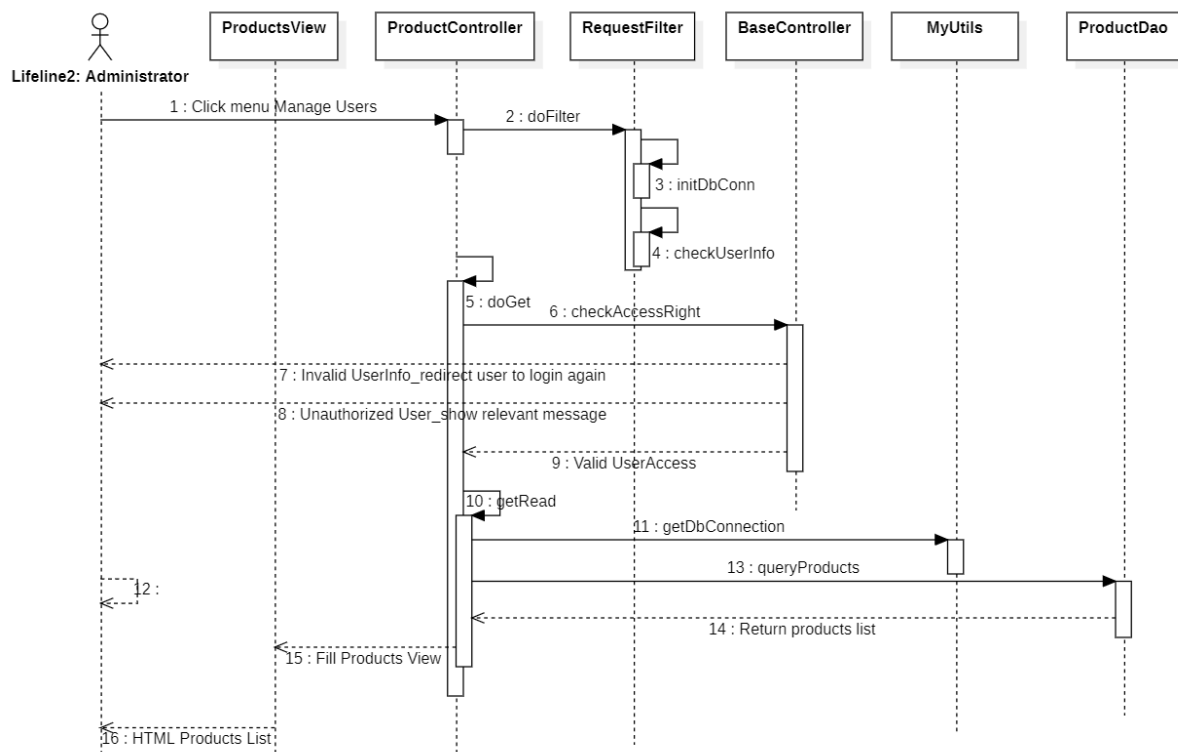
Class Methods

[Provide the detailed description for the class methods]

No	Method	Description
01	<method name>	<Description of the method, including the inputs, outputs & internal method processing>

c. Sequence Diagram(s)

[Provide the sequence diagram(s) for the feature, see the sample below]

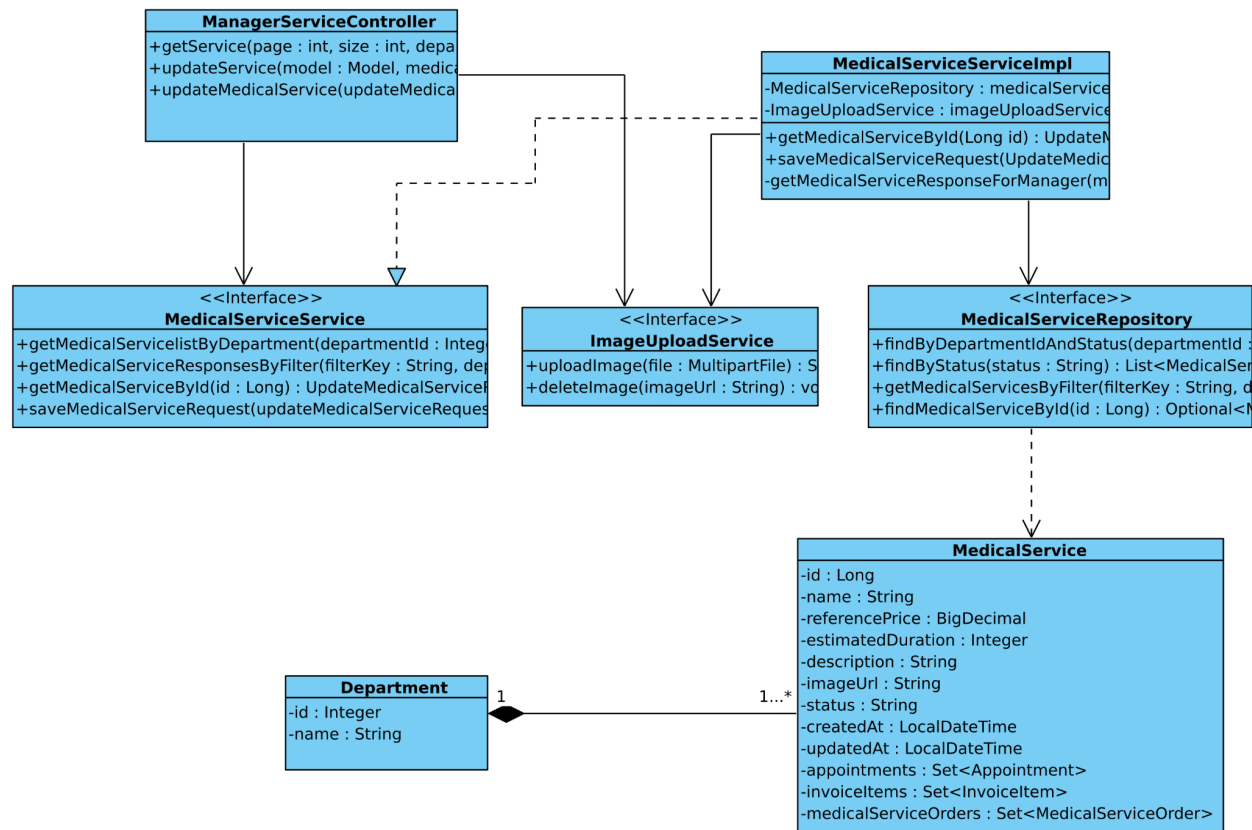


d. Database Queries

[Provide the detailed SQL (select, insert, update...) which are used in implementing the function/screen]

5. Feature Service Management- Manager

a. Class Diagram



b. Class Specifications

Manager ServiceController Class Provide the detailed description for the class methods

Manager ServiceController Class

No	Method	Description
01	getService	Inputs: page (int), size (int), departmentId (Integer), filterKey (String) Outputs: ModelAndView Internal method processing: Gets a paginated list of medical services based on search keywords and department filters. Returns the list to the view.
02	updateService (GET)	Inputs: model (Model), serviceId (Long) Outputs: String (View name) Internal method processing: Fetches the details of a specific medical service by its ID and shows the update form.

03	updateMedicalService(POST)	Inputs: req (UpdateMedicalServiceRequest), bindingResult (BindingResult), model (Model), redirectAttributes (RedirectAttributes), file (MultipartFile) Outputs: String (Redirect URL) Internal method processing: Receives form data and a new image file. If a file is uploaded, it saves the image. Then, it saves the updated service information and redirects back with a success message.
----	----------------------------	--

MedicalServiceService Interface Provide the detailed description for the class methods

MedicalServiceService Interface

No	Method	Description
01	getMedicalServiceListByDepartment	Inputs: departmentId (Integer) Outputs: List<MedicalService> Internal method processing: A rule to find all active medical services that belong to a specific department.
02	getMedicalServiceResponsesByFilter	Inputs: filterKey (String), departmentId (Integer), page (int), size (int) Outputs: Page<MedicalServiceResponseForManager> Internal method processing: A rule to search for medical services using a keyword and department filter, then returns a paginated list.
03	getMedicalServiceById	Inputs: id (Long) Outputs: UpdateMedicalServiceRequest Internal method processing: A rule to find a single medical service by its ID.
04	saveMedicalServiceRequest	Inputs: req (UpdateMedicalServiceRequest) Outputs: UpdateMedicalServiceRequest Internal method processing: A rule to save the updated medical service information.

MedicalServiceServiceImpl Class Provide the detailed description for the class methods

MedicalServiceServiceImpl Class

No	Method	Description
01	getMedicalServiceResponsesByFilter	Inputs: filterKey (String), departmentId (Integer), page (int), size (int) Outputs: Page<MedicalServiceResponseForManager> Internal method processing: Formats the search keyword and calls the repository to search. It then converts the results into a DTO format for the manager view.
02	getMedicalServiceById	Inputs: id (Long) Outputs: UpdateMedicalServiceRequest Internal method processing: Finds the service in the database and prepares its data to be shown on the update form. Throws an error if not found.
03	saveMedicalServiceRequest	Inputs: req (UpdateMedicalServiceRequest) Outputs: UpdateMedicalServiceRequest Internal method processing: Finds the old service in the database, updates its information (price, description, duration). If a new image is provided, it deletes the old image and saves the new one. Finally, it saves the changes to the database.
04	getMedicalServiceResponseForManager	Inputs: medicalService (MedicalService) Outputs: MedicalServiceResponseForManager Internal method processing: A helper method that copies data from a MedicalService entity into a response DTO.

ImageUploadService Interface Provide the detailed description for the class methods

ImageUploadService Interface

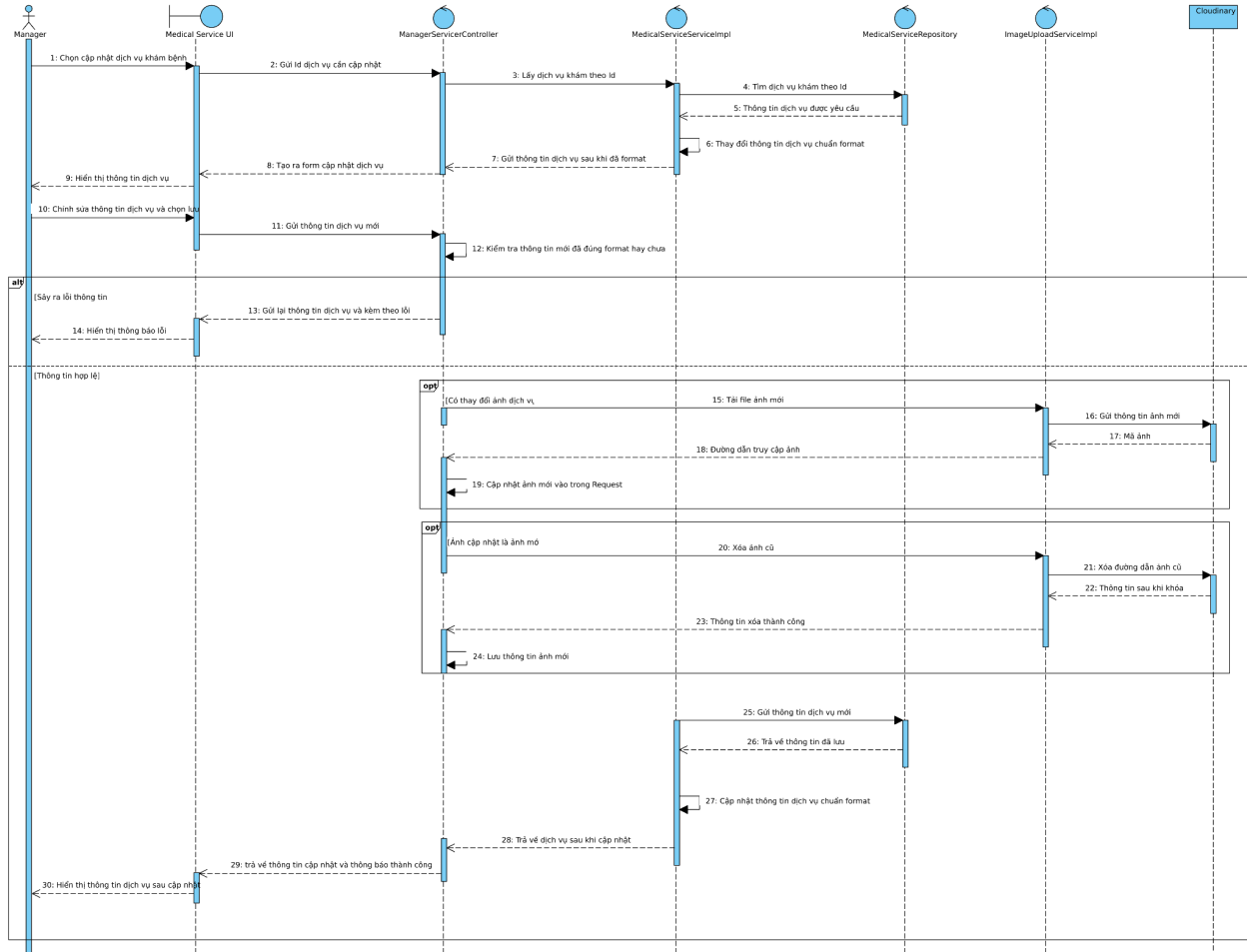
No	Method	Description
01	uploadImage	Inputs: file (MultipartFile) Outputs: String (Image URL) Internal method processing: Uploads the image file to the cloud server and returns the public URL to view it.
02	deleteImage	Inputs: imageUrl (String) Outputs: void Internal method processing: Extracts the image ID from the URL and deletes it from the cloud server.

MedicalServiceRepository Interface Provide the detailed description for the class methods

MedicalServiceRepository Interface

No	Method	Description
01	getMedicalServicesByFilter	Inputs: filterKey (String), departmentId (Integer), pageable (Pageable) Outputs: Page<MedicalService> Internal method processing: Runs a custom database query to search for services by name and department, returning paginated results.
02	findMedicalServiceById	Inputs: id (Long) Outputs: Optional<MedicalService> Internal method processing: Runs a query to find exactly one medical service by its ID, including its department information.

c. Sequence Diagram(s)



d. Database Queries

[Provide the detailed SQL (select, insert, update...) which are used in implementing the function/screen]